

Cardano SeedSigner: Air-Gapped Hardware Wallet Application Specification

Document Version:	1.0
Author:	Angel Castillo
Date Created:	2026/01/07

Table of Contents

Table of Contents.....	2
Overview.....	4
Project Overview.....	4
Relationship to Bitcoin SeedSigner.....	4
Trust Boundaries.....	5
User Consent and Authorization.....	5
Hardware Architecture.....	6
Supported Hardware.....	6
Hardware Compatibility Matrix.....	6
Hardware Interaction Model.....	7
Button Input.....	7
Display Output.....	7
Camera Scanning.....	8
Optional MicroSD Usage.....	8
Power and Lifecycle.....	9
Stateless Operation.....	9
Memory Clearing on Power Loss.....	9
Application Architecture.....	10
Application Structure.....	10
State Management.....	11
In-Memory Keys Only.....	11
No Persistent Secrets.....	12
User Interface & Menus.....	13
User Interaction Principles.....	13
Explicit User Consent.....	13
Human-Verifiable Information.....	13
Main Menu Overview.....	14
Supported Main Menu Options.....	14
Scan Flow.....	14
Supported Scan Inputs.....	14
Seed Management.....	15
Supported Seed Operations.....	15
Tools Menu.....	16
Supported Tools.....	16
Transaction Signing Flow.....	16
Message Signing Flow.....	17
Settings & Device Configuration.....	17
General Settings.....	17

Hardware Settings.....	17
Error and Warning Screens.....	19
Cardano Key & Signing Model.....	20
Supported Key Types.....	20
Derivation & Identification.....	20
Fingerprints / Key Identification.....	21
QR Communication & UR Encoding Model.....	22
Communication Pattern.....	22
UR URI Format.....	22
Encoding Building Blocks.....	23
UR Types Used by Cardano SeedSigner.....	24
Example: Seed import (seed).....	24
Example: Account public key export request (cardano-account-req).....	25
Example: Account public key export (cardano-account).....	25
Example: Cardano Transaction Signing Request (cardano-tx-sig-req).....	25
Example: Cardano Signed Transaction Response (cardano-cip8-sig-res).....	26
Example: Message Signing Request.....	26
Example: Cardano Signed Transaction Response (cardano-cip8-sig-res).....	26
Example: Settings (settings).....	27
Fragmentation & Animated QR.....	27
Compatibility & Extensibility.....	28
User & System Interaction Diagrams.....	29
Seed Import.....	29
Extended Public Key (xpub) Export.....	30
Settings Import/Export.....	31
Transaction Signing.....	32
CIP-8 Message Signing.....	33

Overview

Project Overview

Cardano SeedSigner is a DIY, fully air-gapped hardware wallet application designed to enable secure key management, transaction signing, and message signing for the Cardano blockchain using animated QR codes.

The project extends the existing SeedSigner execution model to Cardano, preserving its stateless operation, offline security guarantees, and user-mediated authorization flow while adapting it to Cardano-specific transaction semantics, signing requirements, and data structures.

This document defines the technical architecture, user experience flows, and QR-based communication protocols required to support Cardano while maintaining SeedSigner's core security principles.

Relationship to Bitcoin SeedSigner

Cardano SeedSigner is based on the Bitcoin SeedSigner project, which serves as the primary architectural reference for this work. Bitcoin SeedSigner demonstrates a fully offline signing device that relies exclusively on animated QR codes for data transfer and enforces a strict separation between offline signing and online transaction construction.

This project reuses SeedSigner's proven hardware abstractions, stateless execution model, and minimal trusted computing base. Where necessary, the design adapts these concepts to Cardano-specific requirements, such as different key roles, transaction structures, and signing contexts, without introducing new cryptographic primitives or transport mechanisms.

See <https://seedsigner.com/>

Trust Boundaries

The online wallet software is treated as untrusted. The device does not assume the correctness of host-provided metadata such as UTxO ownership, balances, or transaction intent beyond what can be cryptographically verified or clearly presented to the user.

User Consent and Authorization

The user experience is designed around explicit user consent rather than inferred intent. Every signing operation requires direct user approval based on information shown on the device itself. The device does not autonomously infer ownership or intent and does not rely on unverifiable claims from the host.

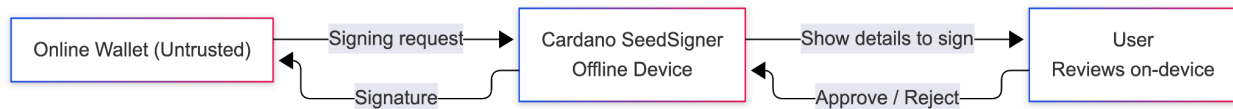


Figure 1 Transaction/Message sign flow.

This approach ensures that authorization decisions remain user-driven, inspectable, and resistant to manipulation by compromised or malicious online software.

The user is also able to export public data to the outside world, example:



Figure 2 Exporting XPUB.

Hardware Architecture

Supported Hardware

Cardano SeedSigner is designed to run on commodity, widely available hardware with no proprietary components. The reference platform is the Raspberry Pi family, selected for its open ecosystem, strong community support, and compatibility with the existing SeedSigner hardware model.

Supported configurations include:

- Raspberry Pi Zero class devices and newer
- SPI-connected color LCD displays
- Raspberry Pi Camera Module (CSI interface)
- Physical GPIO buttons for all user input
- Optional MicroSD card for non-sensitive settings storage

The hardware design deliberately excludes networking interfaces, secure elements, and persistent secret storage. All cryptographic material exists only in volatile memory for the duration of a powered session.

Hardware Compatibility Matrix

The following table summarizes the Raspberry Pi platforms and display combinations that will be supported:

Raspberry Pi Model	GPIO Header	Display Support	Camera Support
Pi Zero / Zero W	40-pin	ST7789	v1.3 / v2
Pi Zero 2 W	40-pin	ST7789	v1.3 / v2
Pi 3 (all variants)	40-pin	ST7789 / ILI9341	v1.3 / v2
Pi 4 (all variants)	40-pin	ST7789 / ILI9341	v1.3 / v2

Hardware Interaction Model

The device interacts with the user and the external environment exclusively through a small, well-defined set of hardware interfaces. These interfaces are used only for local input and output and never for network communication.

Button Input

All user input is performed via physical GPIO buttons, arranged as a 5-way directional pad with additional action buttons.

Button inputs are configured as active-low GPIO signals using internal pull-up resistors. All critical actions, including transaction approval, key management, and navigation, require explicit physical interaction. No remote or automated input paths exist.

Representative GPIO pin assignments:

Pin	Function
31	Up
35	Down
29	Left
37	Right
33	Select
40	Back
38	Power

Display Output

The display is a SPI-connected color LCD and serves as a security-critical output surface. All information required for user verification, addresses, transaction details, stake actions, warnings, and QR codes, is rendered directly on the device.

Supported displays include:

Controller	Resolution	SPI Speed
ST7789	240×240 / 320×240	40 MHz
ILI9341	240×320	64 MHz

Displays operate using 16-bit RGB565 color mode over SPI bus 0. The display driver abstraction allows multiple panels to be supported with consistent rendering behavior.

Camera Scanning

The camera is used solely for scanning QR codes. These QR codes represent structured requests from the online wallet, such as transaction signing requests or configuration payloads.

Camera input is treated as untrusted until decoded, validated, and presented to the user for approval. No automatic actions are taken based on scanned data without explicit user consent.

Optional MicroSD Usage

An optional MicroSD card may be used for non-sensitive device settings.

The MicroSD interface is explicitly **not** used for seed storage or private key persistence. Removal of the MicroSD card does not impact the security model of the device.

Power and Lifecycle

Stateless Operation

Cardano SeedSigner operates in a strictly stateless manner. All sensitive data, including private keys and derived signing material, exists only in volatile memory for the duration of the device's powered-on session.

The device does not persist keys, seeds, or signing state across reboots.

Memory Clearing on Power Loss

Loss of power results in the immediate destruction of all in-memory secrets. No recovery mechanism exists for restoring keys after shutdown, reboot, or unexpected power interruption.

This lifecycle model ensures that physical possession of the powered-off device does not grant access to sensitive material and reinforces the air-gapped security posture of the system.

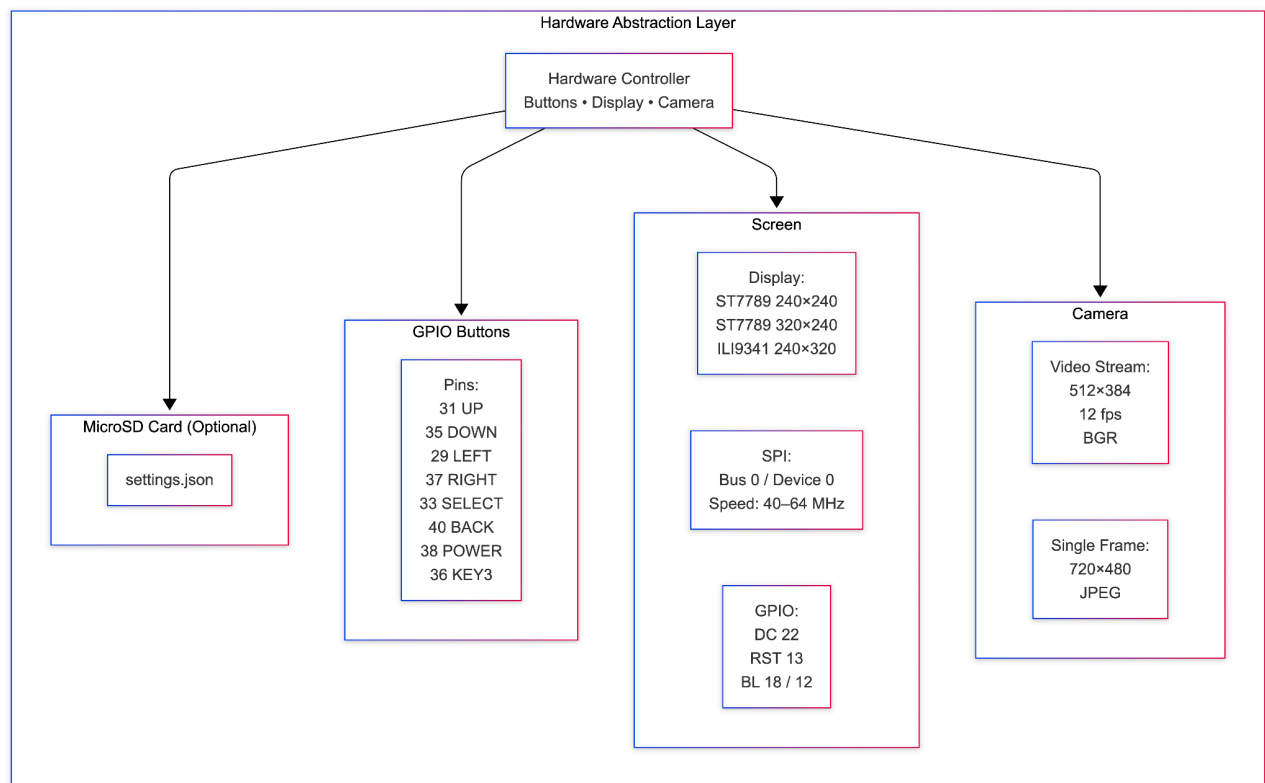


Figure 3 Hardware Abstraction Layer of Cardano SeedSigner.

Application Architecture

Application Structure

Cardano SeedSigner follows a simple, modular application architecture designed to minimize complexity and reduce the trusted computing base. The application is structured around a small number of well-defined components responsible for user interaction, state handling, and cryptographic operations.

At a high level, the system is organized into:

- **Views and screens**, which render information to the display and collect user input
- **Controllers**, which coordinate flows such as scanning, signing, and key management
- **Service components**, which perform cryptographic operations and QR encoding/decoding

User-facing functionality is expressed as explicit flows composed of multiple screens. Each flow is linear, interruptible, and requires direct user confirmation at all authorization boundaries. There is no background processing that can advance application state without user interaction.

This structure mirrors the original SeedSigner architecture, enabling reuse of proven design patterns while allowing Cardano-specific signing and data models to be introduced where required.

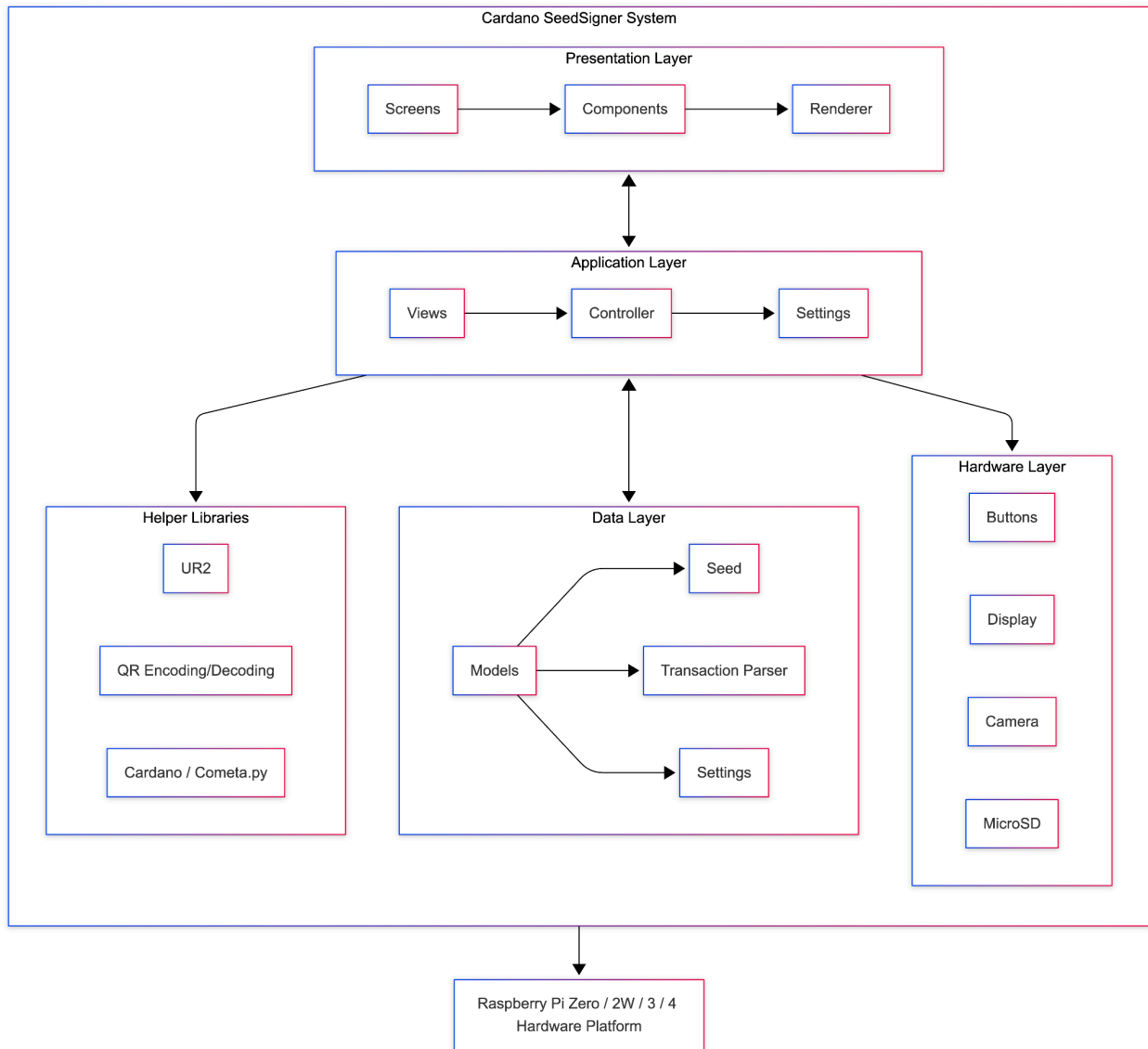


Figure 4 High-level application architecture of Cardano SeedSigner.

State Management

In-Memory Keys Only

All sensitive cryptographic material, including seeds, private keys, and derived signing keys, is held exclusively in volatile memory. Keys are loaded into memory only when explicitly provided by the user and remain available solely for the duration of the current session.

The application does not serialize, cache, or checkpoint private material to disk, removable media, or non-volatile storage.

No Persistent Secrets

Cardano SeedSigner does not persist secrets across application restarts or power cycles. When the device is shut down, restarted, or loses power, all in-memory state is destroyed.

Persistent storage, when present, is limited to non-sensitive configuration such as user interface preferences. The application enforces a strict separation between configuration state and cryptographic state to prevent accidental leakage or misuse of sensitive data.

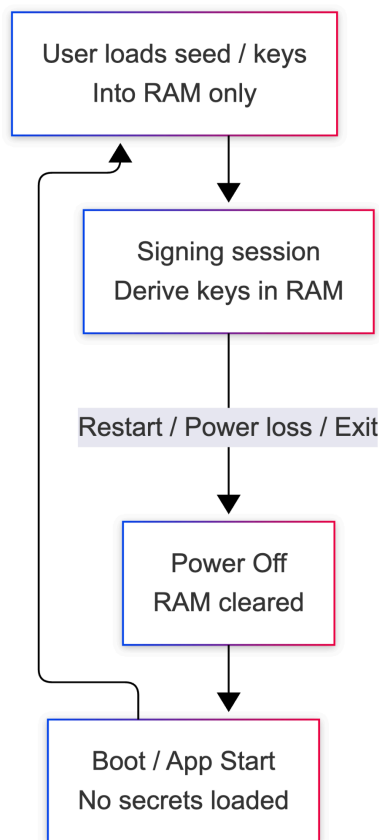


Figure 5 Seed / keys life cycle.

User Interface & Menus

User Interaction Principles

The Cardano SeedSigner user interface is intentionally minimal and security-driven. All interactions are performed locally on the device using physical controls and a trusted display.

Explicit User Consent

All security-relevant actions, such as loading a seed, signing a transaction, or exporting public information, require explicit user approval. The device never performs signing operations automatically or based on inferred intent.

There is no background processing, cached authorization, or unattended operation.

Human-Verifiable Information

Authorization decisions are based solely on information rendered on the device's screen. The interface prioritizes the display of verifiable data such as addresses, amounts, transaction hashes, and derivation paths. The device does not rely on labels, summaries, or metadata supplied by the online wallet unless they can be independently verified or clearly presented to the user.

Main Menu Overview

The main menu is the entry point for all device functionality and exposes only a limited, security-reviewed set of operations.

Supported Main Menu Options

Menu Option	Description
Scan	Universal QR scanner for signing requests and xpub export requests
Seed	Seed loading and seed creation entry point
Tools	Utilities for seed generation and address inspection
Settings	Device configuration and hardware settings
Power Off	Safe shutdown screen

The menu structure is deliberately flat to minimize navigation depth and reduce the likelihood of user error during critical operations.

Scan Flow

The Scan option provides a universal QR ingestion interface. All supported QR payloads are detected and routed based on content type.

Supported Scan Inputs

QR Type	Resulting Action
Signing request	Transaction or message signing flow
Xpub export request	Xpub derivation and display
SeedQR	Seed loading flow

Scanning a QR code never triggers an automatic action. All decoded information is presented for user review before any further processing occurs.

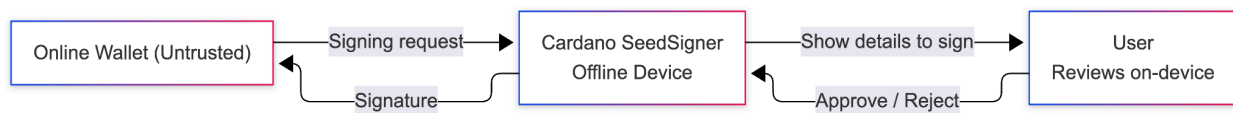


Figure 6 Transaction/Message sign flow.

Seed Management

Seed material is managed explicitly and remains in volatile memory only for the duration of the session.

Supported Seed Operations

Operation	Description
Scan a SeedQR	Import a QR-encoded seed
Enter 12-word seed	Manual BIP-39 entry
Enter 24-word seed	Manual BIP-39 entry
Create a seed	Redirects to Tools menu

Once loaded, a seed can be used for signing operations during the active session. Seeds can be discarded at any time, immediately removing them from memory.

Tools Menu

The Tools menu provides offline utilities that do not require interaction with an online wallet.

Supported Tools

Tool	Description
New Seed (Camera)	Generate entropy using camera image capture
New Seed (Dice)	Generate entropy using physical dice rolls
Calculate 12th / 24th Word	Compute BIP-39 checksum word
Address Explorer	Derive and inspect addresses from a loaded seed

These tools are designed to support fully offline key generation and verification workflows.

Transaction Signing Flow

The transaction signing flow is linear and review-driven.

1. A signing request is scanned via QR
2. The transaction structure is parsed
3. Relevant transaction details are displayed
4. The user explicitly approves or rejects the request
5. A signature or signed transaction is output as animated QR codes

The device does not assume ownership of inputs or correctness of balances beyond what is presented and cryptographically verifiable.

Message Signing Flow

Message signing follows the same authorization model as transaction signing.

The user is shown:

1. The message content
2. The signing context (key or derivation path)

Signing proceeds only after explicit confirmation, and the resulting signature is displayed as a QR code.

Settings & Device Configuration

The Settings menu exposes only non-sensitive configuration options.

General Settings

Setting	Description
Language	Interface language (only English will be available at start)
QR Density	Animated QR frame density
Camera Rotation	Camera orientation
Persist Settings	Enable or disable settings persistence
I/O Test	Button and display test

Hardware Settings

Setting	Description
Display Type	ST7789 (240×240), ST7789 (320×240), ILI9341 (320×240)
Invert Colors	Enabled / Disabled

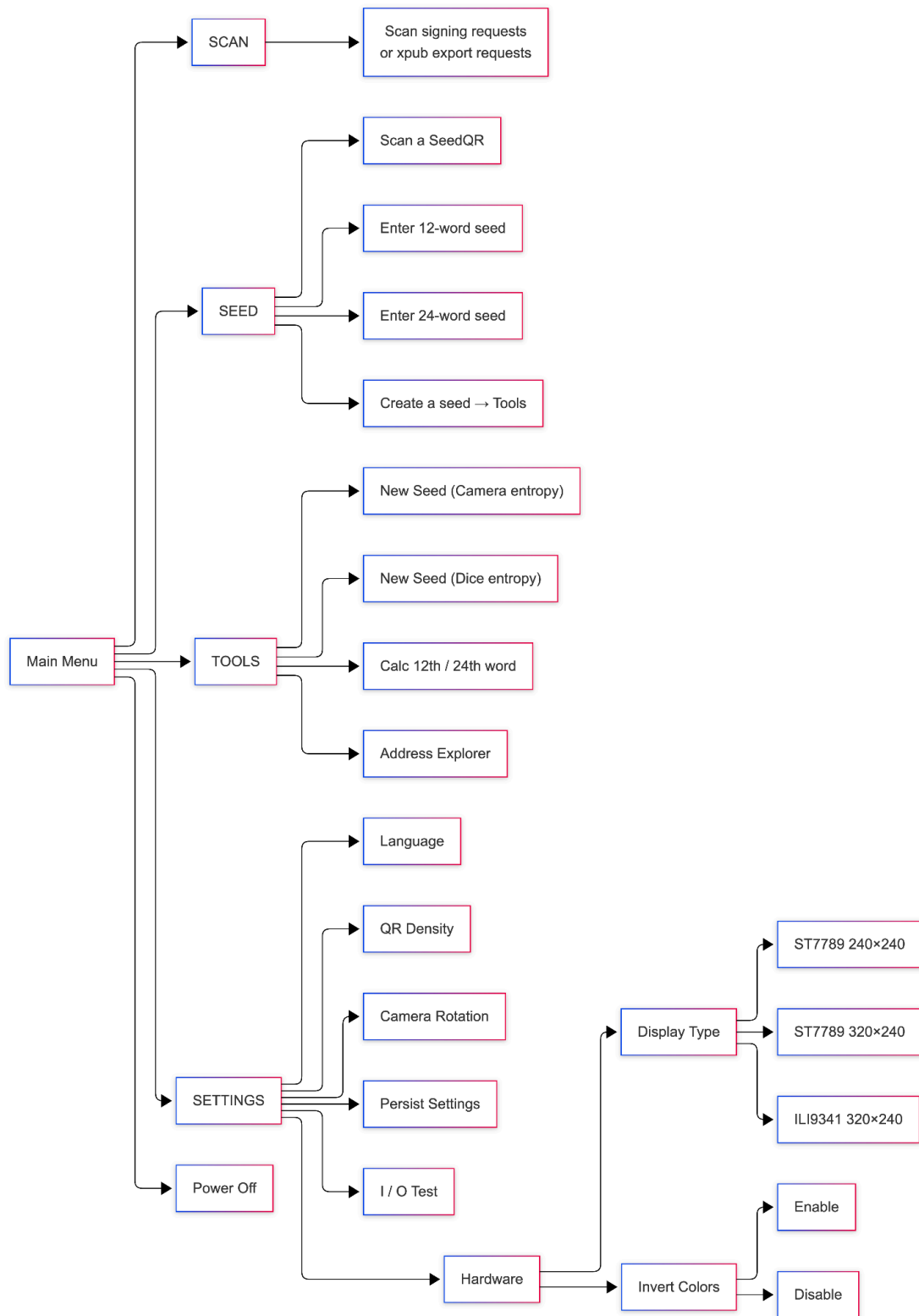


Figure 7 Menu tree.

Error and Warning Screens

The device includes dedicated error and warning screens to handle exceptional conditions such as invalid QR codes, unsupported requests, or hardware failures.

Errors are presented clearly and require user acknowledgment before returning to the main menu. No automatic retries or fallback actions are performed.

Cardano Key & Signing Model

This section defines which Cardano keys the device supports, how they are derived and identified, and what it means for the user to authorize a signing operation on an air-gapped device.

Supported Key Types

Cardano SeedSigner operates exclusively on keys derived from the Ed25519 elliptic curve, using Cardano's hierarchical deterministic wallet scheme commonly referred to as Ed25519-BIP32. Unlike Bitcoin's secp256k1 curve, Ed25519 provides deterministic signatures (EdDSA) and is paired in Cardano with a hardened-only derivation model. This means that all private key derivations use hardened paths, and extended public keys cannot be used to derive child private keys or public keys beyond their intended scope.

Within this model, Cardano distinguishes between several functional key types derived from the same root seed. Payment keys are used to authorize spending of UTxOs. Stake keys are used for delegation, reward withdrawal, and governance-related actions where applicable. Additional specialized roles, such as governance representative (DRep) keys, are derived from the same hierarchical structure but serve distinct purposes in transaction authorization and on-chain participation. All signatures produced by the device are EdDSA signatures over Ed25519 and are valid only within the Cardano protocol context.

Derivation & Identification

Cardano wallets use BIP-39 mnemonics as the user-facing representation of entropy. The mnemonic, optionally combined with a passphrase, is converted into a binary seed using the standard BIP-39 key-stretching process. From this seed, Cardano SeedSigner derives keys using Cardano's registered derivation hierarchy, as defined by CIP-1852 and related specifications.

At a high level, Cardano derivation paths follow the structure:

```
m / 1852' / 1815' / account' / role / index
```

Where:

- 1852' identifies the Cardano HD wallet scheme,
- 1815' is Cardano's registered coin type,

- account' separates independent wallet accounts,
- role distinguishes payment, stake, and other key roles,
- index selects a specific key within that role.

For example:

A payment receiving key may be derived at:

```
m / 1852' / 1815' / 0' / 0 / 0
```

A stake key is derived at:

```
m / 1852' / 1815' / 0' / 2 / 0
```

SeedSigner uses these derivation paths only to derive keys required for the specific operation being authorized.

Fingerprints / Key Identification

Cardano SeedSigner computes a short, deterministic fingerprint for each loaded seed in order to provide a stable, human-verifiable identifier throughout the user session. The fingerprint is used exclusively for user orientation and confirmation and does not participate in any cryptographic authorization or signing decisions.

When a seed is loaded, the device first derives the Cardano root key material from the user-provided BIP-39 mnemonic, applying the standard BIP-39 key-stretching process and any optional passphrase configured by the user. From this root, the device derives a canonical account-level key using Cardano's hierarchical deterministic structure, following the CIP-1852 derivation path (m / 1852' / 1815' / account'). This level is chosen because it uniquely represents a wallet account while remaining independent of individual payment or stake keys.

The fingerprint itself is derived solely from public key material associated with this account. The public key is serialized in a deterministic form and passed through a cryptographic hash function. A short prefix of the resulting hash is then taken and displayed as a compact hexadecimal identifier (e.g., A1B2C3D4). This truncation is sufficient to reliably distinguish between seeds in the user interface while keeping the value readable and easy to compare.

QR Communication & UR Encoding Model

Cardano SeedSigner uses animated QR codes exclusively, encoded using UR2 (Blockchain Commons Uniform Resources). UR is a transport format designed specifically for camera scanning: it wraps CBOR-encoded binary payloads into a QR-safe string using bytewords, and when the payload is large it uses fountain codes to split it into a stream of recoverable frames.

This section defines the request/response model, the UR frame format, the UR types used, and provides representative examples (including Cardano-specific signing requests).

Communication Pattern

All interactions follow a strict unidirectional flow at each step:

1. Host → Device: Request encoded as UR
2. User → Device: Visual inspection and explicit approval
3. Device → Host: Response encoded as UR

The device treats all incoming data as untrusted and relies on user review and cryptographic verification before producing any output.

UR URI Format

A UR is represented as:

```
ur:<type>[/<sequence>]/<body>
```

- ur: protocol prefix (lowercase)
- <type> payload type (e.g., crypto-seed, crypto-account)
- <sequence> optional seqnum-seqlen for multipart animated URs
- <body> bytewords-encoded CBOR (minimal encoding)

Single-part example:

```
ur:crypto-cardano-tx/hdeymejtswhhylkepmykhhtsytsnoyoyaxaedsuttydmmhhpktpmsrjtgdwpfns  
boxgwlbaawzuefywkdpjrsrjnbvygabwjldapfcsdwkbrkch
```

Multipart example:

```
ur:crypto-cardano-tx/2-5/lpaoaocsptcybgksksajkinjkcxinjkcxhscxjtjljyihjkjycxjnihjyhsjnieihjphsj  
zihkthsieiejzhsjniaishsiahsjy
```

Notes:

- seq_len is the count of unique fragments.
- seq_num can exceed seq_len once the encoder starts emitting redundant fountain frames.

Encoding Building Blocks

CBOR (RFC 8949) is used to represent structured data compactly. UR types commonly use CBOR **semantic tags** to identify standardized structures (e.g., seed, hdkey, account).

Bytewords converts bytes into text suitable for QR scanning. UR uses minimal bytewords, encoding each byte into 2 characters (first+last letters of a 4-letter word), plus a CRC checksum.

Fountain codes (LT codes) enable robust animated scanning:

- The sender can emit unlimited frames,
- The receiver can miss frames,
- And still recover the original message after collecting roughly $\sim 1.75\times$ the unique fragment count in typical conditions.

UR Types Used by Cardano SeedSigner

Cardano SeedSigner reuses existing UR types where applicable (seed), and introduces Cardano-specific signing request/response types.

Purpose	UR Type	CBOR Tag	Direction
Seed import	seed	40300	Incoming
Account public key export request	cardano-account-req	88004	Incoming
Account public key	cardano-account	88005	Outgoing
settings payloads	settings	88006	Both
Cardano transaction body signing	cardano-tx-sig-req	88000	Incoming
Cardano message signing	cardano-cip8-sig-req	88001	Incoming
Cardano transaction body signing result	cardano-tx-sig-res	88002	Outgoing
Cardano message signing result	cardano-cip8-sig-res	88003	Outgoing

Example: Seed import (seed)

```
ur:seed/oadgdstasplabghydrpfmkbggufgludprfgmaotpiecffltlnlpqdenos
```

Payload

```
{
  1: h'c7098580125e2ab0981253468b2dbc52',
  2: 100(18394)
}
```

See

<https://github.com/BlockchainCommons/Research/blob/master/papers/bcr-2020-006-urtypes.md#cryptographic-seed-seed> for more details

Example: Account public key export request (cardano-account-req)

ur:cardano-account-req/oeadcyemrewytyaolttantjyoeadiojojeisdefzdydtaolytan

Payload

```
{
  1: <account_index>,
  2: <key_purpose>
}
```

Example: Account public key export (cardano-account)

ur:cardano-account-req/oeadcyemrewytyaolttantjyoeadiojojeisdefzdydtaolytan

Payload

```
{
  1: <xpub_hex>
}
```

Example: Cardano Transaction Signing Request (cardano-tx-sig-req)

ur:cardano-tx-sig-req/1-4/lpadaocsptcybgkskgsajk...

Payload (CBOR, conceptual):

```
{
  1: h'<tx_body_cbor>',
  2: [<required_signing_paths>],
  3: [<Resolved Inputs>]
}
```

Example: Cardano Signed Transaction Response (cardano-cip8-sig-res)

ur:cardano-tx-sig-res/lpadaocsptcybgksksajk...

Payload (CBOR, conceptual):

```
{
  1: h'<vkey_witness_set_cbor>',
}
```

Example: Message Signing Request

ur:cardano-cip8-sig-req/lpaoacsptcyaxhdcxjih...

Payload (CBOR, conceptual):

```
{
  1: h'<message_payload>',
  3: h'<address_bytes>' or null
  2: <required_signing_path>
}
```

Example: Cardano Signed Transaction Response (cardano-cip8-sig-res)

ur:cardano-cip8-sig-res/lpadaocsptcybgksksajk...

Payload (CBOR, conceptual):

```
{
  1: h'<cose_sign1_cbor>',
  2: h'<cose_key_cbor>',
}
```

Example: Settings (settings)

```
ur:settings/lpadaocsptcybgksksajk...
```

Payload (CBOR, conceptual):

```
{
  1: <language>,
  2: <display_brightness>,
  3: <qr_density>,
  4: <camera_rotation>,
  5: <display_type>,
  4: <invert_colors>,
}
```

Fragmentation & Animated QR

UR payloads exceeding a single QR frame are automatically fragmented using fountain codes. This provides:

- Loss tolerance: Frames may be skipped or scanned out of order
- Progressive decoding: Completion without all frames
- User control: Scanning can be interrupted safely at any time

Each UR frame includes:

- A sequence identifier
- Payload fragment
- CRC checksum for integrity

```
ur:cardano-tx-sig-req/3-7/lpaxaycsptcybgksk...
```

Compatibility & Extensibility

Cardano SeedSigner follows established UR conventions to ensure interoperability. Where Cardano-specific payloads are required (e.g., transaction and message signing), new UR types are defined without modifying the underlying UR transport or encoding rules.

Future extensions may introduce additional Cardano UR types while preserving backward compatibility and the same animated QR communication model.

User & System Interaction Diagrams

Seed Import

The user initiates the flow directly on the device by scanning a SeedQR or manually entering mnemonic words. The seed is validated locally, loaded into volatile memory, and made available for signing operations during the current session only. No external actor is involved, and the seed material is never transmitted back to the hot wallet.

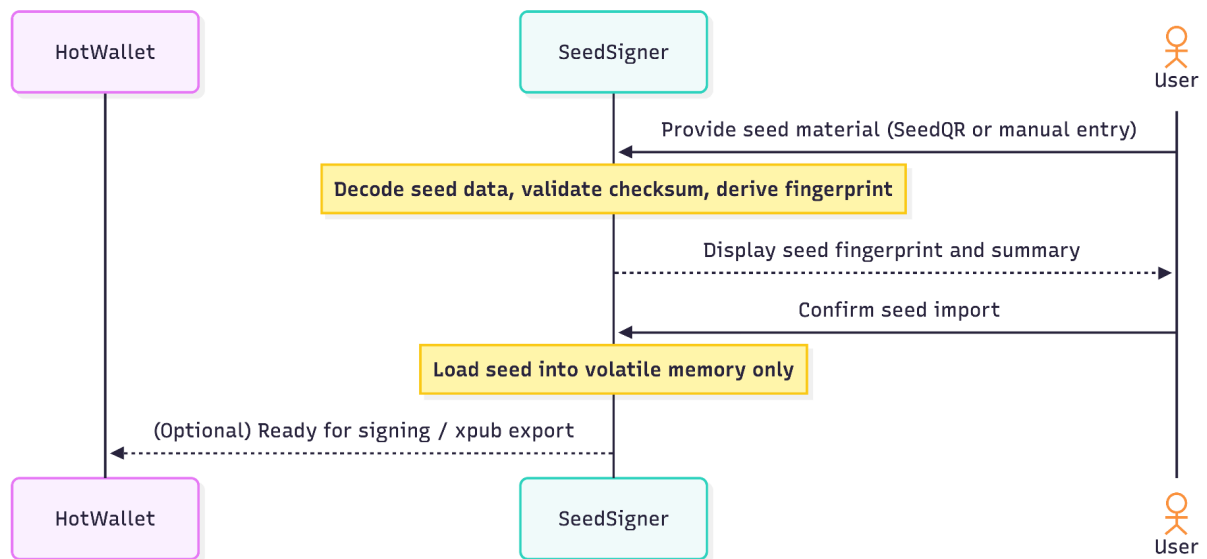


Figure 8 Seed import flow.

Extended Public Key (xpub) Export

After the user selects an export option, SeedSigner derives the requested extended public key according to the specified Cardano derivation path. The derived public data is encoded as a QR payload and displayed to the user. The hot wallet scans the QR to import the xpub for watch-only or account setup purposes. Private key material never leaves the device.

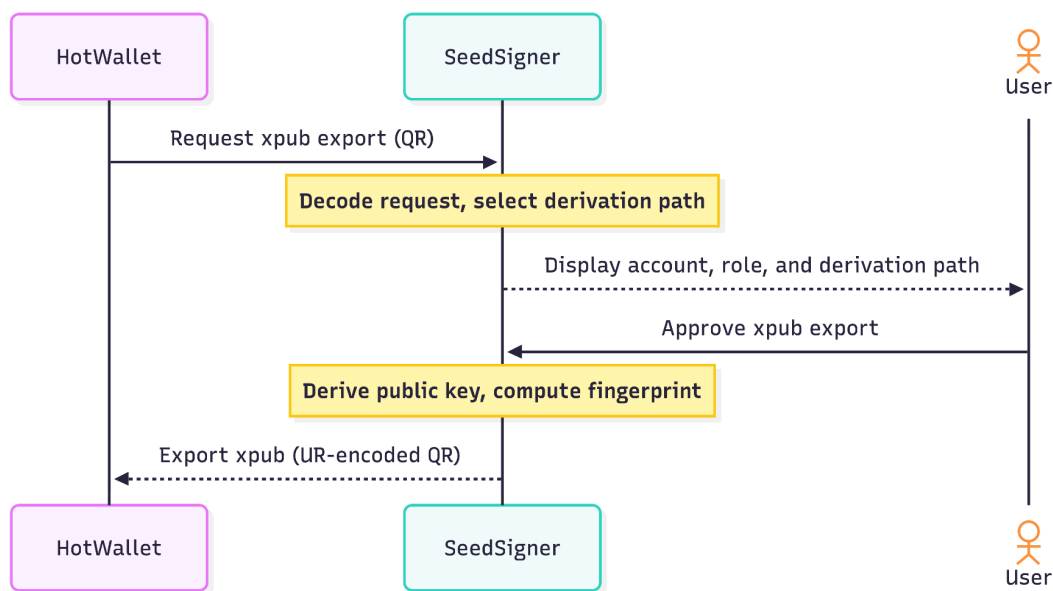


Figure 9 Extended public key export flow.

Settings Import/Export

The user initiates the export from the settings menu, after which SeedSigner serializes only configuration state such as language, QR density, or hardware preferences. The exported settings are presented as a QR code that can be scanned by a hot wallet or companion application. Cryptographic material is explicitly excluded from this process.

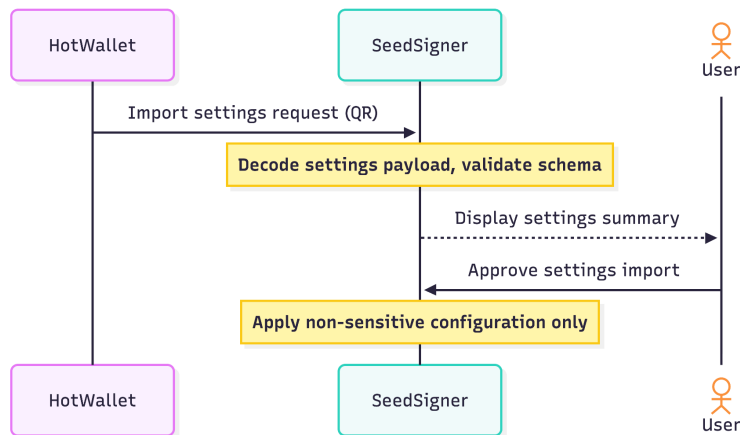


Figure 10 Settings import flow.

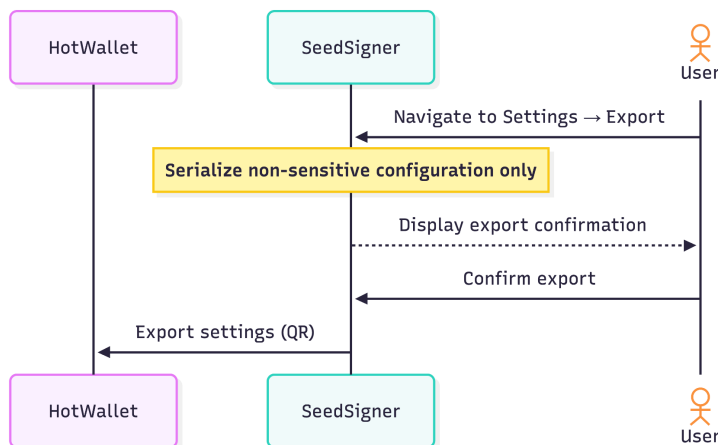


Figure 11 Settings export flow.

Transaction Signing

The hot wallet submits a transaction signing request to SeedSigner using a QR-encoded payload. SeedSigner decodes and validates the request, then presents a human-verifiable summary of the transaction to the user. Signing proceeds only after explicit user approval, and the resulting signed transaction is returned to the hot wallet via QR code.

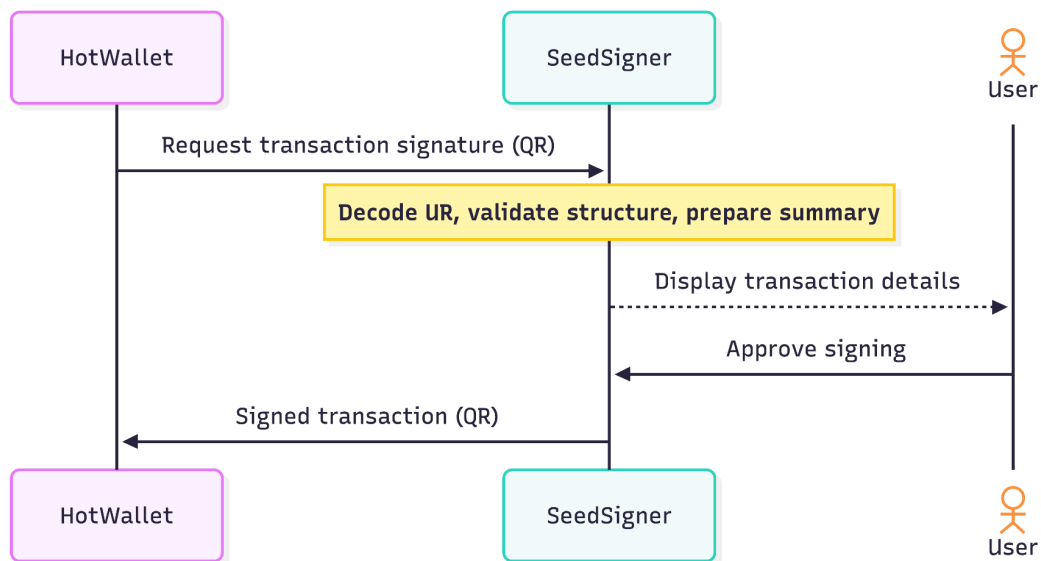


Figure 12 Transaction signing flow.

CIP-8 Message Signing

A hot wallet submits a message signing request to SeedSigner using a QR-encoded payload that specifies the message and signing context. SeedSigner decodes the request, displays the full message content and relevant derivation information to the user, and waits for explicit approval. Upon confirmation, SeedSigner derives the appropriate key and produces an Ed25519 signature compliant with CIP-8, returning the signature to the hot wallet via QR code.

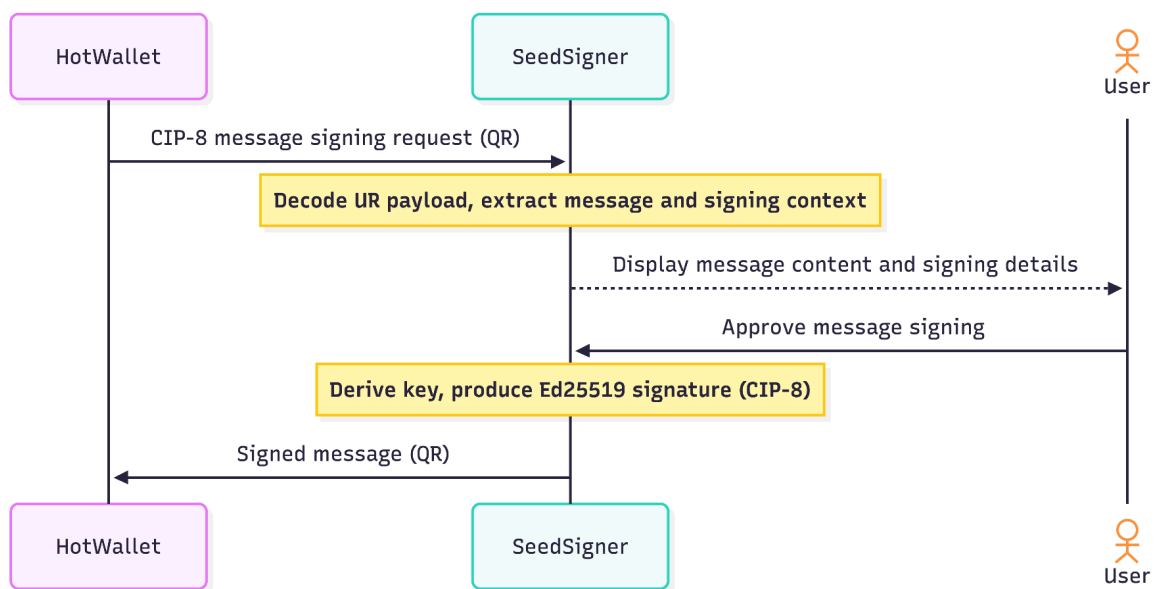


Figure 13 CIP-8 Message signing flow.

